

# Alicia Grechu

Los Angeles, California | aliciagrechu@gmail.com | [linkedin.com/in/alicia-grechu/](https://www.linkedin.com/in/alicia-grechu/) | [aliciagrechu.github.io](https://github.com/aliciagrechu)

## EDUCATION

**UNIVERSITY OF SOUTHERN CALIFORNIA, Los Angeles, CA**  
Bachelor of Science, Computer Engineering and Computer Science

**Expected Grad: May 2028**  
Video Game Programming Minor

## SKILLS

**C++, C#, Python, Java, Verilog | Unreal Engine (C++ & Blueprints), Unity | Perforce, Git/GitHub | Agile/Scrum**

## WORK EXPERIENCE

**Fear Elements Studio, Remote Internship**

**Jun. 2026-Present**

*Gameplay Programmer & Programming Lead*

- **Own** the player movement, health, and HUD systems in **Unreal Engine (C++ and Blueprints)**, including 3 penguin locomotion modes (waddling, swimming, tobogganing) with distinct physics and state transitions
- Lead technical direction for a **6-person gameplay programming team**, setting coding standards, reviewing pull requests, and breaking down feature work into manageable tasks and sprints
- Balance hands-on implementation of gameplay features with mentoring fellow programmers and **coordinating across design and production** to keep development on schedule

## RELEVANT PROJECTS

**Crease (Gameplay Engineer - Team Project) | Unity/C#, Perforce**

**Jun. 2026-Present**

- **Own** the WindSplineTube system in **Unity/C#**, which guides the player along 3D splines using tangent-based flow and a radial centering force, with reversible direction and particle VFX
- Iterated on the centering force through 3 rounds of playtesting to balance guidance with player control, so players are guided by the wind but can always escape the current if desired

**Frogger (Gameplay Programmer - Solo Project) | C++, GitHub**

**Sep. 2026**

- Built a Frogger clone from scratch in **C++ with no engine**, on a custom framework developed across multiple games: a hybrid actor/component model (inheritance plus component composition), actor lifecycle management with destruction deferred until after the update loop, and a draw-order sprite renderer with a texture cache
- Implemented AABB collision with minimum-overlap resolution, verified with **unit tests** on GitHub Actions, plus a text-file level loader that spawns actors from a tile grid

**Large Language Mimic (Gameplay Programmer - Team Project) | Unity/C#, GitHub**

**Jan. 2026-May 2026**

- Programmed core gameplay mechanics, NPC behavior, and UI feedback in **Unity/C#** for a satirical 2D game where players impersonate an AI chatbot, with branching dialogue and multiple endings
- Implemented character animations and scene sequencing that hands off cleanly between minigames (e.g. file redaction, photo cleanup), integrating gameplay systems across a **25-person programming team**

## ACTIVITIES & INVOLVEMENT

**USC Open Alpha - Game Programmer & Vice President of Tech**

**Sep. 2025-Present**

- Lead technical direction for the club's semesterly **Unity/C#** game as VP of Tech: set up and maintain the GitHub repository, project configs, and builds, and manage releases on itch.io and Steam
- Developed and teach a **Unity/C#** curriculum to onboard new programmers to the team
- Previously **shipped 2 games** as a gameplay programmer; work with interdisciplinary teams (design, programming, art, sound) to design mechanics, implement features, conduct playtests, and iterate fast based on feedback

**USC Makers - Software Engineer**

**Sep. 2025-May 2026**

- Built **FIFO**, a smart fridge system designed to reduce household food waste by tracking food inventory and expiration dates using sensors, barcode scanning, and computer vision, working across an **8-person engineering team**
- Developed a **Detectron2 computer vision pipeline** trained on **200+ personally annotated images** in COCO format, cropping detected containers for VLM identification and migrating inference to a cloud GPU to overcome a **20-30 minute local processing bottleneck**
- Created the **Unity/C#** companion app's expiration tracking, streak, and avatar customization systems, integrating them with **Firestore** and a **Flask/Python API** to keep inventory synchronized across the product